VEGGITY DEVELOPER HANDBOOK

Complete Frontend Architecture & Code Guide

A step-by-step master walkthrough for building the Hyperlocal Vegetable Marketplace UI in React + Vite: From Location Autocomplete to Freshness Engines and Real-Time Feeds.

Project:	**Veggity (Hyperlocal Perishable Marketplace)**
Frontend Tech:	**React 18, Vite, OpenStreetMap Nominatim, React Toastify**
Focus:	**Produce Listing Form, GPS Detection, Debounced Search & Discovery Feed**
Skill Level:	**Beginner to Professional (Line-by-Line Breakdown)**

1. The Big Picture: Frontend Architecture & Structure

The Veggity frontend is designed around two user roles:

1. **The Seller (Gardener/Farmer):** Wants to list fresh surplus vegetables with safe expiry dates and their village location in less than 30 seconds.
2. **The Buyer (Hungry Neighbor):** Wants to see fresh, unexpired food growing within a 5 km – 10 km radius of their current location.

> 💡 **Design Principle: Reusable Lego Blocks**
> Notice that both the *Seller* (setting farm location) and the *Buyer* (browsing local produce) need to search villages or detect their GPS. Rather than duplicating 150 lines of code, we build a single reusable component: `<LocationSearch />`.

📁 Complete Frontend File Hierarchy

```
frontend/
├── index.html                  # HTML Entry Point
├── package.json                # Dependencies (react, react-router-dom, react-toastify)
├── vite.config.js              # Vite build configuration
└── src/
    ├── App.jsx                 # Master Route Switch (/login, /signup, /home, /add-produce)
    ├── App.css                 # Global App Layout Styles
    ├── index.css               # Base Resets & Colors
    ├── main.jsx                # React root renderer
    ├── utils.jsx               # Shared API_URL & Toast notification helpers
    │
    ├── components/
    │   ├── LocationSearch.jsx  # 🌟 REUSABLE: OpenStreetMap Autocomplete + GPS Auto Detect
    │   └── LocationSearch.css  # Dropdown, spinner, pin icon, clean styling
    │
    └── pages/
        ├── Login.jsx & Login.css       # Existing Auth Login Page
        ├── Signup.jsx & Signup.css     # Existing Auth Registration Page
        ├── AddProduce.jsx              # 🌟 SELLER: Form with dynamic shelf-life limits
        ├── AddProduce.css              # Clean cards, form groups, input highlights
        ├── Home.jsx                    # 🌟 BUYER: Discovery Feed with freshness badges
        └── Home.css                    # Toolbar, radius filters, responsive produce grid
```

---

2. Core Frontend Technical Concepts Explained

A. Debouncing (Preventing API Spam)

When a user types "*Rampur*", the browser fires 6 keystrokes (R → a → m → p → u → r). If we make an HTTP request on every key, we send **6 unnecessary requests** to OpenStreetMap.

- **The Fix (Debounce 350ms):** Whenever the user types a letter, we start a 350ms timer. If they type another letter within 350ms, we cancel the old timer and start a new one. We only call the API when the user pauses typing for 350ms!

B. OpenStreetMap Nominatim (Free Geocoding & Reverse Geocoding)

Google Maps API requires a credit card and bills you every month. Veggity uses **OpenStreetMap's Nominatim API**, which is 100% free and open-source:

API Type	Endpoint URL	Input → Output
Forward Geocoding (Autocomplete)	`https://nominatim.openstreetmap.org/search?q={query}&countrycodes=in`	Text "*Rampur*" → Coordinates `[79.1845, 28.7934]`
Reverse Geocoding (Auto-Detect GPS)	`https://nominatim.openstreetmap.org/reverse?lat={lat}&lon={lng}`	GPS numbers → Friendly Village Name "*Rampur, Uttar Pradesh*"

C. Dynamic Shelf-Life Engine Matching Backend Categories

Your backend enforces strict shelf-life caps based on vegetable biology. The frontend mirrors these caps dynamically:

Category Key	Display Label	Max Cap	Why?
`Leafy Vegetables`	🥬 Leafy Greens (Palak, Methi)	**2 Days**	Soft leaves wither and rot rapidly.
`Fruit Vegetables`	🍅 Fruit Veggies (Tomatoes, Okra, Brinjal)	**4 Days**	Skin breaks down, loses firmness.
`Root Vegetables`	🥕 Root Veggies (Carrots, Radish)	**15 Days**	Hard roots stay fresh for 1–2 weeks.
`Bulb Vegetables`	🧅 Bulb Veggies (Onions, Garlic)	**15 Days**	Dry outer skin protects moisture.
`Herbs`	🌿 Fresh Herbs (Mint, Coriander, Lemon)	**5 Days**	Aromatic oils dry out.
`Other Vegetables`	🥦 Other Vegetables (Beans, Gourds)	**5 Days**	General safe buffer.

D. Dynamic Freshness Badges

We calculate the remaining life in real time by subtracting `Date.now()` from `item.expiresAt`:

- If `diffHours <= 0`: **Expired**
- If `diffHours < 24`: ⏰ **6h fresh left** (Urgent warning badge)

- If `diffHours >= 24`: 🌿 **2d fresh remaining** (Fresh badge)

---

3. File 1: LocationSearch Component

`frontend/src/components/LocationSearch.jsx`

This is the core geolocation widget used by both the seller and buyer pages.

```
import React, { useState, useEffect, useRef } from 'react';
import './LocationSearch.css';

function LocationSearch({
    onLocationSelect,
    initialName = '',
    placeholder = 'Search village, town, or locality name...',
    showAutoDetect = true
}) {
    const [query, setQuery] = useState(initialName);
    const [suggestions, setSuggestions] = useState([]);
    const [isOpen, setIsOpen] = useState(false);
    const [isLoading, setIsLoading] = useState(false);
    const [isDetectingGps, setIsDetectingGps] = useState(false);
    const [statusMessage, setStatusMessage] = useState('');

    const wrapperRef = useRef(null);

    // Sync query with initialName prop updates
    useEffect(() => {
        if (initialName) setQuery(initialName);
    }, [initialName]);

    // Close suggestions dropdown when clicking anywhere outside
    useEffect(() => {
        function handleClickOutside(event) {
            if (wrapperRef.current && !wrapperRef.current.contains(event.target)) {
                setIsOpen(false);
            }
        }
        document.addEventListener('mousedown', handleClickOutside);
        return () => document.removeEventListener('mousedown', handleClickOutside);
    }, []);

    // Debounced search using OpenStreetMap Nominatim API
    useEffect(() => {
        if (!query || query.trim().length < 2) {
            setSuggestions([]);
            setIsLoading(false);
            return;
        }

        setIsLoading(true);
        const timer = setTimeout(async () => {
            try {
                const url = `https://nominatim.openstreetmap.org/search?
format=json&q=${encodeURIComponent(query.trim())}&limit=6&addressdetails=1&countrycodes=in`;
                const response = await fetch(url, {
                    headers: { 'Accept-Language': 'en' }
                });
                const data = await response.json();

                if (Array.isArray(data)) {
```

```
                    const formatted = data.map((item) => {
                        const addr = item.address || {};
                        const primaryName = addr.village || addr.town || addr.city || addr.suburb ||
addr.neighbourhood || addr.county || item.name;
                        const stateOrCountry = addr.state || addr.country || '';
                        const displayName = primaryName && stateOrCountry ? `${primaryName},
${stateOrCountry}` : item.display_name;

                        return {
                            id: item.place_id,
                            displayName: displayName,
                            fullAddress: item.display_name,
                            lat: parseFloat(item.lat),
                            lng: parseFloat(item.lon)
                        };
                    });
                    setSuggestions(formatted);
                    setIsOpen(true);
                }
            } catch (err) {
                console.error('Location search error:', err);
                setSuggestions([]);
            } finally {
                setIsLoading(false);
            }
        }, 350);

        return () => clearTimeout(timer);
    }, [query]);

    // Handle user selecting an autocomplete suggestion
    const handleSelect = (item) => {
        setQuery(item.displayName);
        setIsOpen(false);
        setStatusMessage(`📍 Location set: ${item.displayName}`);
        if (onLocationSelect) {
            onLocationSelect({
                name: item.displayName,
                fullAddress: item.fullAddress,
                lat: item.lat,
                lng: item.lng
            });
        }
    };

    // Auto Detect GPS and Reverse-Geocode coordinates
    const handleAutoDetect = () => {
        if (!navigator.geolocation) {
            setStatusMessage('⚠️ Geolocation not supported by your browser');
            return;
        }

        setIsDetectingGps(true);
        setStatusMessage('🎯 Detecting your GPS location...');

        navigator.geolocation.getCurrentPosition(
            async (position) => {
                const lat = position.coords.latitude;
                const lng = position.coords.longitude;

                try {
                    const revUrl = `https://nominatim.openstreetmap.org/reverse?
format=json&lat=${lat}&lon=${lng}&addressdetails=1`;
```

```
                    const res = await fetch(revUrl, {
                        headers: { 'Accept-Language': 'en' }
                    });
                    const data = await res.json();

                    const addr = data.address || {};
                    const locName = addr.village || addr.suburb || addr.town || addr.city ||
addr.neighbourhood || 'Nearby Location';
                    const state = addr.state || '';
                    const friendlyName = state ? `${locName}, ${state}` : locName;

                    setQuery(friendlyName);
                    setStatusMessage(`📍 Auto-detected: ${friendlyName}`);

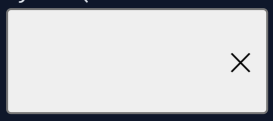
                    if (onLocationSelect) {
                        onLocationSelect({
                            name: friendlyName,
                            fullAddress: data.display_name || friendlyName,
                            lat: lat,
                            lng: lng
                        });
                    }
                } catch (err) {
                    console.error('Reverse geocode error:', err);
                    const fallbackName = `Current Location (${lat.toFixed(4)}, ${lng.toFixed(4)})`;
                    setQuery(fallbackName);
                    setStatusMessage(`📍 GPS Captured`);
                    if (onLocationSelect) {
                        onLocationSelect({ name: fallbackName, fullAddress: fallbackName, lat, lng });
                    }
                } finally {
                    setIsDetectingGps(false);
                }
            },
            (err) => {
                console.error(err);
                setIsDetectingGps(false);
                setStatusMessage('⚠️ Location permission denied. Please type your location.');
            },
            { enableHighAccuracy: true, timeout: 10000 }
        );
    };

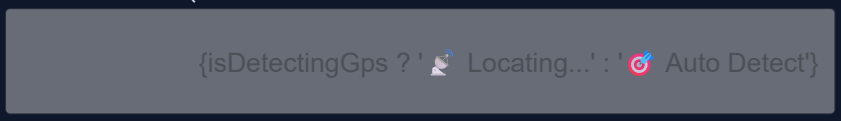
    const handleClear = () => {
        setQuery('');
        setSuggestions([]);
        setStatusMessage('');
        setIsOpen(false);
    };

    return (




                    🔍
                    {query}                    {
                            setQuery(e.target.value);
                            setStatusMessage('');
                        }}
                        placeholder={placeholder}
```

```
                        onFocus={() => {
                            if (suggestions.length > 0) setIsOpen(true);
                        }}
                    />
                    {query && (
                        
                    )}
                    {isLoading &&
}


                {showAutoDetect && (
                    
                )}


            {isOpen && suggestions.length > 0 && (

                    {suggestions.map((item) => (

  • handleSelect(item)}>
                                

                                    {item.displayName}
                                    {item.fullAddress}

                    ))}

            )}

            {statusMessage && (

{statusMessage}

            )}

    );
}

export default LocationSearch;
```

Line-by-Line Logic Breakdown of LocationSearch

- **Props (`onLocationSelect`):** Function passed by parent. When a village is picked, it receives `{ name, lat, lng }`.
- **`wrapperRef` and `handleClickOutside`:** Uses React's `useRef` to check if the user clicks anywhere outside this input box. If so, it automatically hides the suggestion dropdown.

- `setTimeout(..., 350)`: The debounce timer. `return () => clearTimeout(timer)` clears the old timer whenever the user types again within 350 milliseconds.
- `navigator.geolocation.getCurrentPosition`: Native browser API that queries the hardware GPS sensor. It returns `coords.latitude` and `coords.longitude`.
- **Reverse Geocoding:** Calls Nominatim's `/reverse` endpoint with the coordinates to find the friendly village/town name instead of showing ugly numbers to the user.

LocationSearch CSS Styling

`frontend/src/components/LocationSearch.css`

```
.location-search-wrapper {
    position: relative;
    width: 100%;
    font-family: inherit;
}

.location-input-row {
    display: flex;
    gap: 8px;
    align-items: center;
}

.search-input-box {
    position: relative;
    flex: 1;
    display: flex;
    align-items: center;
}

.search-icon {
    position: absolute;
    left: 12px;
    font-size: 14px;
    color: #94a3b8;
    pointer-events: none;
}

.location-text-input {
    width: 100%;
    padding: 10px 36px 10px 36px;
    font-size: 13.5px;
    border: 1.5px solid #cbd5e1;
    border-radius: 8px;
    outline: none;
    transition: all 0.2s ease;
    background: #ffffff;
    color: #0f172a;
}

.location-text-input:focus {
    border-color: #10b981;
    box-shadow: 0 0 0 3px rgba(16, 185, 129, 0.15);
}

.btn-clear-search {
    position: absolute;
    right: 10px;
    background: transparent;
    border: none;
    color: #94a3b8;
    font-size: 14px;
    cursor: pointer;
    padding: 4px;
}

.btn-clear-search:hover {
    color: #ef4444;
```

```
}

.btn-auto-gps {
    padding: 10px 14px;
    background: #ecfdf5;
    color: #065f46;
    border: 1.5px solid #a7f3d0;
    border-radius: 8px;
    font-size: 12.5px;
    font-weight: 700;
    cursor: pointer;
    white-space: nowrap;
    transition: all 0.2s;
    display: flex;
    align-items: center;
    gap: 6px;
}

.btn-auto-gps:hover:not(:disabled) {
    background: #d1fae5;
    border-color: #6ee7b7;
}

.btn-auto-gps:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}

.suggestions-dropdown {
    position: absolute;
    top: calc(100% + 4px);
    left: 0;
    right: 0;
    background: #ffffff;
    border: 1.5px solid #e2e8f0;
    border-radius: 8px;
    box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.1);
    max-height: 220px;
    overflow-y: auto;
    z-index: 1000;
    margin: 0;
    padding: 4px 0;
    list-style: none;
}

.suggestion-item {
    display: flex;
    align-items: flex-start;
    gap: 10px;
    padding: 8px 12px;
    cursor: pointer;
    border-bottom: 1px solid #f1f5f9;
    transition: background 0.15s;
}

.suggestion-item:last-child {
    border-bottom: none;
}

.suggestion-item:hover {
    background: #f0fdf4;
}
```

```
.sugg-pin {
    font-size: 15px;
    margin-top: 2px;
}

.sugg-text {
    display: flex;
    flex-direction: column;
}

.sugg-name {
    font-size: 13px;
    color: #0f172a;
}

.sugg-full {
    font-size: 11px;
    color: #64748b;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
    max-width: 380px;
}

.location-status-text {
    font-size: 11.5px;
    color: #059669;
    font-weight: 600;
    margin-top: 4px;
    padding-left: 2px;
}

.input-spinner {
    position: absolute;
    right: 32px;
    width: 14px;
    height: 14px;
    border: 2px solid #e2e8f0;
    border-top-color: #10b981;
    border-radius: 50%;
    animation: spin 0.6s linear infinite;
}

@keyframes spin {
    to { transform: rotate(360deg); }
}
```

---

4. File 2: AddProduce Page (Seller Listing Form)

`frontend/src/pages/AddProduce.jsx`

This is the farmer/gardener form where homegrown produce is listed. It connects to `POST /products/add`.

```
import React, { useState, useEffect } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { ToastContainer } from 'react-toastify';
import { API_URL, handleError, handleSuccess } from '../utils';
import LocationSearch from '../components/LocationSearch';
import './AddProduce.css';

// 6 Categories matching your backend Product.js enum & ProductController.js limits
const CATEGORY_CONFIG = {
    'Leafy Vegetables': { maxDays: 2, defaultDays: 1, name: 'Leafy Greens (Palak, Methi)' },
    'Fruit Vegetables': { maxDays: 4, defaultDays: 2, name: 'Fruit Veggies (Tomatoes, Okra)' },
    'Root Vegetables':  { maxDays: 15, defaultDays: 7, name: 'Root Veggies (Carrots, Radish)' },
    'Bulb Vegetables':  { maxDays: 15, defaultDays: 7, name: 'Bulb Veggies (Onions, Garlic)' },
    'Herbs':            { maxDays: 5, defaultDays: 2, name: 'Fresh Herbs (Mint, Coriander, Lemon)' },
    'Other Vegetables': { maxDays: 5, defaultDays: 3, name: 'Other Vegetables (Beans, Gourds)' }
};

function AddProduce() {
    const navigate = useNavigate();
    const [loggedInUser, setLoggedInUser] = useState('');
    const [loading, setLoading] = useState(false);
    const [selectedLocationName, setSelectedLocationName] = useState('');

    const [formData, setFormData] = useState({
        title: '',
        category: 'Leafy Vegetables',
        shelfLifeDays: '1',
        price: '',
        unit: 'kg',
        quantity: '',
        locationArea: '',
        latitude: null,
        longitude: null
    });

    // Protected Route Check: Only logged in users can list produce
    useEffect(() => {
        const token = localStorage.getItem('token');
        const user = localStorage.getItem('loggedInUser');
        if (!token) {
            handleError('Please login to list produce');
            navigate('/login');
            return;
        }
        setLoggedInUser(user || 'Neighbor');
    }, [navigate]);

    // Handle general text/number input changes
    const handleChange = (e) => {
        const { name, value } = e.target;
        setFormData((prev) => ({ ...prev, [name]: value }));
    };
```

```
    // When category changes, reset shelfLifeDays to the safe default for that category
    const handleCategoryChange = (e) => {
        const newCat = e.target.value;
        const defaultShelf = CATEGORY_CONFIG[newCat]?.defaultDays || 1;
        setFormData((prev) => ({
            ...prev,
            category: newCat,
            shelfLifeDays: String(defaultShelf)
        }));
    };

    // Callback when user picks village in
    const handleLocationSelect = ({ name, lat, lng }) => {
        setSelectedLocationName(name);
        setFormData((prev) => ({
            ...prev,
            latitude: lat,
            longitude: lng,
            locationArea: prev.locationArea || name
        }));
    };

    // Submit form to backend POST /products/add
    const handleSubmit = async (e) => {
        e.preventDefault();

        const { title, category, price, unit, quantity, shelfLifeDays, locationArea, latitude, longitude } =
formData;

        // Validation checks
        if (!title.trim() || !price || !quantity || !locationArea.trim()) {
            return handleError('Please fill in all required fields');
        }

        if (latitude === null || longitude === null) {
            return handleError('Please search and select your village/area using the location search');
        }

        const token = localStorage.getItem('token');
        setLoading(true);

        try {
            const response = await fetch(`${API_URL}/products/add`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': token
                },
                body: JSON.stringify({
                    title: title.trim(),
                    category,
                    price: Number(price),
                    unit,
                    quantity: Number(quantity),
                    shelfLifeDays: Number(shelfLifeDays),
                    locationArea: locationArea.trim(),
                    latitude: Number(latitude),
                    longitude: Number(longitude)
                })
            });

            const result = await response.json();
```

```
            if (result.success) {
                handleSuccess('Produce listed successfully!');
                setTimeout(() => navigate('/home'), 1200);
            } else {
                handleError(result.message || 'Failed to list produce');
            }
        } catch (err) {
            handleError(err.message || 'Network error while listing produce');
        } finally {
            setLoading(false);
        }
    };

    const currentCatInfo = CATEGORY_CONFIG[formData.category] || CATEGORY_CONFIG['Leafy Vegetables'];

    return (
```

Veggity
Seller Portal

Grower: **{loggedInUser}**
View Market

Share your garden harvest with neighbors within a 5km–10km radius before it spoils.

Vegetable Name / Title *

{formData.title}

Category *

{config.name}

Freshness Shelf-Life (Days) *

{day} {day === 1 ? 'Day' : 'Days'} {day === currentCatInfo.defaultDays ? '(Recommended)' : ''}

ℹ️ **Safety Rule:** {currentCatInfo.name} stays fresh for maximum **{currentCatInfo.maxDays} days.** Once this time passes, it automatically vanishes from the neighborhood feed.

Price (₹) *

e.g. 40

Unit

kg

Available Quantity *

e.g. 5

📍 Garden / Farm Location *

Type your village, town, or city name — suggestions will appear as you type. Or click Auto Detect!

```
                        {selectedLocationName && (

                                ✅ Location locked: {selectedLocationName}

                        )}




                        Neighborhood Landmark / Street Address *
                        {formData.locationArea}
                        Helps nearby neighbors find your exact pickup spot easily.

                                {loading ? 'Listing Produce...' : '🌱 List Produce for Sale'}

    );
}

export default AddProduce;
```

AddProduce CSS Styling

`frontend/src/pages/AddProduce.css`

```
.add-produce-page {
    min-height: 100vh;
    background: #f8fafc;
    color: #1e293b;
    font-family: inherit;
    padding-bottom: 60px;
}

.page-header {
    background: #ffffff;
    border-bottom: 1.5px solid #e2e8f0;
    padding: 14px 28px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.header-brand {
    display: flex;
    align-items: center;
    gap: 12px;
}

.logo-link {
    font-size: 20px;
    font-weight: 800;
    color: #059669;
    text-decoration: none;
}

.page-title-badge {
    background: #ecfdf5;
    color: #065f46;
    font-size: 11.5px;
    font-weight: 700;
    padding: 3px 8px;
    border-radius: 6px;
    border: 1px solid #a7f3d0;
}

.header-actions {
    display: flex;
    align-items: center;
    gap: 16px;
}

.user-greeting {
    font-size: 13px;
    color: #475569;
}

.btn-secondary {
    padding: 8px 14px;
    background: #ffffff;
    color: #059669;
    border: 1.5px solid #10b981;
    border-radius: 8px;
```

```
    text-decoration: none;
    font-size: 12.5px;
    font-weight: 700;
    transition: all 0.2s;
}

.btn-secondary:hover {
    background: #ecfdf5;
}

.form-wrapper {
    max-width: 680px;
    margin: 32px auto 0 auto;
    padding: 0 16px;
}

.form-card {
    background: #ffffff;
    border: 1.5px solid #e2e8f0;
    border-radius: 12px;
    padding: 32px 36px;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}

.card-header h2 {
    margin: 0 0 6px 0;
    font-size: 20px;
    color: #0f172a;
}

.card-header p {
    margin: 0 0 24px 0;
    font-size: 13px;
    color: #64748b;
}

.form-group {
    display: flex;
    flex-direction: column;
    margin-bottom: 18px;
}

.form-group label {
    font-size: 12.5px;
    font-weight: 700;
    color: #334155;
    margin-bottom: 6px;
}

.form-group input,
.form-group select {
    padding: 10px 14px;
    font-size: 13px;
    border: 1.5px solid #cbd5e1;
    border-radius: 8px;
    outline: none;
    transition: border-color 0.2s;
    background: #ffffff;
}

.form-group input:focus,
.form-group select:focus {
    border-color: #10b981;
```

```
    box-shadow: 0 0 0 3px rgba(16, 185, 129, 0.15);
}

.form-row {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 16px;
}

.form-row-three {
    grid-template-columns: 2fr 1fr 2fr;
}

.info-banner {
    background: #f0fdf4;
    border: 1.5px solid #bbf7d0;
    color: #166534;
    padding: 10px 14px;
    border-radius: 8px;
    font-size: 12px;
    margin-bottom: 18px;
}

.location-box {
    background: #f8fafc;
    border: 1.5px dashed #cbd5e1;
    border-radius: 10px;
    padding: 16px;
    margin-bottom: 18px;
}

.location-box-title {
    font-size: 13px;
    font-weight: 700;
    color: #0f172a;
    display: block;
    margin-bottom: 4px;
}

.location-box-sub {
    font-size: 11.5px;
    color: #64748b;
    margin-bottom: 12px;
}

.selected-loc-badge {
    margin-top: 10px;
    background: #ecfdf5;
    border: 1px solid #a7f3d0;
    color: #065f46;
    padding: 6px 12px;
    border-radius: 6px;
    font-size: 12px;
}

.field-hint {
    font-size: 11px;
    color: #94a3b8;
    margin-top: 4px;
}

.btn-submit {
    width: 100%;
```

```
    padding: 14px;
    background: #059669;
    color: #ffffff;
    border: none;
    border-radius: 8px;
    font-size: 14px;
    font-weight: 700;
    cursor: pointer;
    transition: background 0.2s;
    margin-top: 10px;
}

.btn-submit:hover:not(:disabled) {
    background: #047857;
}

.btn-submit:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}
```

---

5. File 3: Home Page (Buyer Discovery Feed)

`frontend/src/pages/Home.jsx`

The buyer discovery feed connects to `GET /products/nearby`. It features distance radius buttons, category filters, and live perishable freshness countdowns.

```
import React, { useEffect, useState, useCallback } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { ToastContainer } from 'react-toastify';
import { API_URL, handleSuccess, handleError } from '../utils';
import LocationSearch from '../components/LocationSearch';
import './Home.css';

// 6 Category tags matching your backend Product.js enum
const CATEGORY_MAP = {
    'all': 'All Produce',
    'Leafy Vegetables': '🥬 Leafy Greens',
    'Fruit Vegetables': '🍅 Fruit Veggies',
    'Root Vegetables':  '🥕 Root Veggies',
    'Bulb Vegetables':  '🧅 Bulb Veggies',
    'Herbs':            '🌿 Fresh Herbs',
    'Other Vegetables': '🥦 Other Veggies'
};

function Home() {
    const navigate = useNavigate();
    const [loggedInUser, setLoggedInUser] = useState('');
    const [products, setProducts] = useState([]);
    const [loading, setLoading] = useState(true);

    // Hyperlocal Search Filters
    const [radiusKm, setRadiusKm] = useState(5); // Default 5km neighborhood
    const [selectedCategory, setSelectedCategory] = useState('all');
    const [locationLabel, setLocationLabel] = useState('My Neighborhood');

    // Viewer Coordinates (Default: Delhi/NCR fallback coordinates)
    const [viewerCoords, setViewerCoords] = useState({
        lat: 28.6139,
        lng: 77.2090
    });

    // Check user login session
    useEffect(() => {
        const user = localStorage.getItem('loggedInUser');
        setLoggedInUser(user || 'Neighbor');
    }, []);

    // Logout Handler
    const handleLogout = () => {
        localStorage.removeItem('token');
        localStorage.removeItem('loggedInUser');
        handleSuccess('Logged out successfully');
        setTimeout(() => navigate('/login'), 800);
    };

    // Fetch produce from backend GET /products/nearby
    const fetchNearbyProduce = useCallback(async (coords, radius, category) => {
        setLoading(true);
```

```
        try {
            const params = new URLSearchParams({
                latitude: coords.lat,
                longitude: coords.lng,
                radiusKm: radius
            });

            if (category && category !== 'all') {
                params.append('category', category);
            }

            const response = await fetch(`${API_URL}/products/nearby?${params.toString()}`);
            const result = await response.json();

            if (result.success) {
                setProducts(result.products || []);
            } else {
                handleError(result.message || 'Could not fetch nearby produce');
                setProducts([]);
            }
        } catch (err) {
            console.error('Fetch error:', err);
            handleError('Failed to connect to marketplace server');
            setProducts([]);
        } finally {
            setLoading(false);
        }
    }, []);

    // Re-fetch produce whenever coordinates, radius, or category change
    useEffect(() => {
        fetchNearbyProduce(viewerCoords, radiusKm, selectedCategory);
    }, [viewerCoords, radiusKm, selectedCategory, fetchNearbyProduce]);

    // Handle Buyer selecting a different village/town in
    const handleViewerLocationSelect = ({ name, lat, lng }) => {
        setLocationLabel(name);
        setViewerCoords({ lat, lng });
    };

    // Dynamic Freshness Badge Calculation
    const getFreshnessBadge = (expiresAt) => {
        const diffMs = new Date(expiresAt) - new Date();
        const diffHours = Math.round(diffMs / (1000 * 60 * 60));

        if (diffHours <= 0) {
            return { label: 'Expired', class: 'badge-expired' };
        }
        if (diffHours < 24) {
            return { label: `⏰ ${diffHours}h fresh left`, class: 'badge-urgent' };
        }
        const diffDays = Math.floor(diffHours / 24);
        return { label: `🌿 ${diffDays}d fresh remaining`, class: 'badge-fresh' };
    };

    // Simple produce booking simulation
    const handleBookProduce = (item) => {
        handleSuccess(`Order reserved for ${item.title}! The seller (${item.seller?.name || 'Local
Neighbor'}) has been notified.`);
    };

    return (
```

```
🌱 Veggity
📍 {locationLabel} (within {radiusKm} km)

Hi, {loggedInUser}
+ List Produce
Logout

    Your Village / Town / Area:

    Discovery Radius:

        {[5, 10, 15, 25, 50].map((km) => (
                setRadiusKm(km)}
                >
                {km} km
        ))}

{Object.entries(CATEGORY_MAP).map(([key, label]) => (
    setSelectedCategory(key)}
    >
    {label}
))}
```

```
                        Direct from local neighborhood growers • Showing unexpired items within {radiusKm}
km of {locationLabel}


                {loading ? (


Scanning gardens and farms in this area for fresh produce...


                ) : products.length === 0 ? (


🌿


No fresh produce listed nearby


We couldn't find any unexpired vegetables within {radiusKm} km of {locationLabel}.


                                    setRadiusKm(radiusKm < 25 ? 25 : 50)}
                                                                    >
                                    🔍 Expand Radius to {radiusKm < 25 ? '25 km' : '50 km'}


                        + Be the first neighbor to list produce!


                ) : (
```

```
                    {products.map((item) => {
                        const freshness = getFreshnessBadge(item.expiresAt);
                        return (
                                                {CATEGORY_MAP[item.category] || item.category}
                                                {freshness.label}
{item.title}
                                                ₹
                                                {item.price}
                                                /{item.unit}
                                                {item.quantity} {item.unit} available
                                                📍
                                                {item.locationArea}
                                                🧑‍🌾
                                                    Grower: {item.seller?.name || 'Local Neighbor'}
```

```
                                handleBookProduce(item)}
                                                >
                                    🛒 Book / Order Produce
                            );
                        })}

                )}

    );
}

export default Home;
```

Home CSS Styling

`frontend/src/pages/Home.css`

```
.home-market-page {
    min-height: 100vh;
    background: #f8fafc;
    color: #1e293b;
    font-family: inherit;
    padding-bottom: 80px;
}

.market-header {
    background: #ffffff;
    border-bottom: 1.5px solid #e2e8f0;
    padding: 12px 28px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.header-left {
    display: flex;
    align-items: center;
    gap: 14px;
}

.market-logo {
    font-size: 22px;
    font-weight: 800;
    color: #059669;
    text-decoration: none;
}

.location-pill {
    background: #ecfdf5;
    color: #065f46;
    font-size: 12px;
    font-weight: 700;
    padding: 4px 10px;
    border-radius: 9999px;
    border: 1px solid #a7f3d0;
}

.header-right {
    display: flex;
    align-items: center;
    gap: 14px;
}

.welcome-text {
    font-size: 13px;
    color: #475569;
}

.btn-add-produce {
    background: #059669;
    color: #ffffff;
    text-decoration: none;
    font-size: 12.5px;
    font-weight: 700;
```

```
    padding: 8px 16px;
    border-radius: 8px;
    transition: background 0.2s;
}

.btn-add-produce:hover {
    background: #047857;
}

.btn-logout {
    background: transparent;
    border: 1.5px solid #cbd5e1;
    color: #64748b;
    font-size: 12.5px;
    font-weight: 700;
    padding: 7px 14px;
    border-radius: 8px;
    cursor: pointer;
    transition: all 0.2s;
}

.btn-logout:hover {
    border-color: #ef4444;
    color: #ef4444;
}

.discovery-toolbar {
    background: #ffffff;
    border-bottom: 1.5px solid #e2e8f0;
    padding: 20px 28px;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
}

.toolbar-container {
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 24px;
    align-items: end;
    margin-bottom: 16px;
}

.toolbar-section {
    display: flex;
    flex-direction: column;
}

.toolbar-label {
    font-size: 12px;
    font-weight: 700;
    color: #64748b;
    margin-bottom: 6px;
    text-transform: uppercase;
    letter-spacing: 0.04em;
}

.radius-selector {
    display: flex;
    gap: 6px;
}

.radius-btn {
    padding: 9px 14px;
    background: #f1f5f9;
```

```
    border: 1.5px solid #cbd5e1;
    border-radius: 8px;
    font-size: 12px;
    font-weight: 700;
    color: #334155;
    cursor: pointer;
    transition: all 0.15s;
}

.radius-btn:hover {
    background: #e2e8f0;
}

.radius-btn.selected {
    background: #059669;
    border-color: #059669;
    color: #ffffff;
}

.category-filter-bar {
    display: flex;
    gap: 8px;
    overflow-x: auto;
    padding-bottom: 4px;
}

.category-chip {
    padding: 6px 14px;
    background: #f8fafc;
    border: 1.5px solid #e2e8f0;
    border-radius: 9999px;
    font-size: 12px;
    font-weight: 600;
    color: #475569;
    cursor: pointer;
    white-space: nowrap;
    transition: all 0.15s;
}

.category-chip:hover {
    background: #f1f5f9;
}

.category-chip.active {
    background: #ecfdf5;
    border-color: #10b981;
    color: #065f46;
    font-weight: 700;
}

.market-feed {
    max-width: 1200px;
    margin: 32px auto 0 auto;
    padding: 0 24px;
}

.feed-header h2 {
    font-size: 22px;
    color: #0f172a;
    margin: 0 0 4px 0;
}

.results-count {
```

```
    font-size: 14px;
    color: #64748b;
    font-weight: 600;
    margin-left: 8px;
}

.feed-subtitle {
    font-size: 13px;
    color: #64748b;
    margin: 0 0 24px 0;
}

.produce-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 20px;
}

.produce-card {
    background: #ffffff;
    border: 1.5px solid #e2e8f0;
    border-radius: 12px;
    padding: 18px;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.04);
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    transition: transform 0.2s, box-shadow 0.2s;
}

.produce-card:hover {
    transform: translateY(-2px);
    box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.08);
}

.card-top {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 10px;
}

.category-tag {
    font-size: 11px;
    font-weight: 700;
    color: #475569;
    background: #f1f5f9;
    padding: 3px 8px;
    border-radius: 6px;
}

.freshness-badge {
    font-size: 11px;
    font-weight: 700;
    padding: 3px 8px;
    border-radius: 6px;
}

.badge-fresh {
    background: #dcfce7;
    color: #166534;
}
```

```
.badge-urgent {
    background: #ffedd5;
    color: #9a3412;
}

.badge-expired {
    background: #fee2e2;
    color: #991b1b;
}

.produce-title {
    font-size: 16px;
    color: #0f172a;
    margin: 0 0 12px 0;
    font-weight: 700;
}

.price-quantity-row {
    display: flex;
    justify-content: space-between;
    align-items: baseline;
    margin-bottom: 14px;
}

.price-tag {
    font-size: 20px;
    font-weight: 800;
    color: #059669;
}

.currency {
    font-size: 16px;
    margin-right: 2px;
}

.unit {
    font-size: 12px;
    color: #64748b;
    font-weight: 600;
}

.quantity-badge {
    font-size: 11.5px;
    font-weight: 600;
    color: #475569;
    background: #f8fafc;
    border: 1px solid #e2e8f0;
    padding: 2px 8px;
    border-radius: 6px;
}

.produce-meta {
    border-top: 1px solid #f1f5f9;
    padding-top: 10px;
    margin-bottom: 16px;
    display: flex;
    flex-direction: column;
    gap: 6px;
}

.meta-item {
    display: flex;
    align-items: center;
```

```
    gap: 6px;
    font-size: 11.5px;
    color: #64748b;
}

.btn-book {
    width: 100%;
    padding: 10px;
    background: #059669;
    color: #ffffff;
    border: none;
    border-radius: 8px;
    font-size: 12.5px;
    font-weight: 700;
    cursor: pointer;
    transition: background 0.2s;
}

.btn-book:hover {
    background: #047857;
}

.loading-state,
.empty-state {
    text-align: center;
    padding: 60px 20px;
    background: #ffffff;
    border: 1.5px dashed #cbd5e1;
    border-radius: 12px;
}

.empty-icon {
    font-size: 40px;
    margin-bottom: 12px;
}

.empty-state h3 {
    margin: 0 0 6px 0;
    color: #0f172a;
}

.empty-state p {
    color: #64748b;
    font-size: 13px;
    margin-bottom: 20px;
}

.empty-actions {
    display: flex;
    justify-content: center;
    gap: 12px;
}

.btn-expand-radius {
    padding: 9px 16px;
    background: #f1f5f9;
    border: 1.5px solid #cbd5e1;
    border-radius: 8px;
    font-size: 12.5px;
    font-weight: 700;
    color: #334155;
    cursor: pointer;
}
```

```
.btn-first-produce {
    padding: 9px 16px;
    background: #059669;
    color: #ffffff;
    text-decoration: none;
    border-radius: 8px;
    font-size: 12.5px;
    font-weight: 700;
}

.spinner {
    width: 32px;
    height: 32px;
    border: 3px solid #e2e8f0;
    border-top-color: #059669;
    border-radius: 50%;
    margin: 0 auto 12px auto;
    animation: spin 0.7s linear infinite;
}
```

---

6. Files 4 & 5: App Routing & Shared Utilities

File 4: App.jsx (Master Navigation Switch)

`frontend/src/App.jsx`

```
import { Navigate, Route, Routes } from 'react-router-dom';
import './App.css';
import Login from './pages/Login';
import Signup from './pages/Signup';
import Home from './pages/Home';
import AddProduce from './pages/AddProduce'; // Newly added produce page

function App() {
    return (



                {/* Fallback default redirect to /login */}
                } />

                {/* Auth Routes */}
                } />
                } />

                {/* Produce & Marketplace Routes */}
                } />
                } />



    );
}

export default App;
```

File 5: utils.jsx (API Config & Toast Notifications)

`frontend/src/utils.jsx`

```
import { toast } from 'react-toastify';

// Auto-fallback: uses VITE_API_URL or defaults to local Node backend on port 8080
export const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8080';

// Global Success Toast
export const handleSuccess = (msg) => {
    toast.success(msg, {
        position: 'top-right',
        autoClose: 2500,
        hideProgressBar: false,
        closeOnClick: true,
        pauseOnHover: true,
        draggable: true
    });
};
```

```
// Global Error Toast
export const handleError = (msg) => {
    toast.error(msg, {
        position: 'top-right',
        autoClose: 3500,
        hideProgressBar: false,
        closeOnClick: true,
        pauseOnHover: true,
        draggable: true
    });
};
```

---

7. Master Build Order & Pitfalls to Avoid

The 4-Step Build Order

1. **Step 1: Create Location Component**
   Create `frontend/src/components/LocationSearch.jsx` and `LocationSearch.css`.
   Test: Verify that typing a village name shows dropdown recommendations, and clicking "Auto Detect" captures GPS coordinates.
2. **Step 2: Create Seller Form**
   Create `frontend/src/pages/AddProduce.jsx` and `AddProduce.css`.
   Test: Select "Leafy Vegetables" and confirm max shelf life is 2 days. Fill in details and click "List Produce" to create your first MongoDB produce item!
3. **Step 3: Connect Routing**
   Update `frontend/src/App.jsx` to import `AddProduce` and mount `<Route path='/add-produce' element=<AddProduce /> />`.
4. **Step 4: Upgrade Buyer Home Feed**
   Update `frontend/src/pages/Home.jsx` and `Home.css`.
   Test: Confirm your newly listed produce appears in the grid with a green freshness badge (`🌿 2d fresh remaining`). Click different radius buttons (5km, 10km) to see dynamic filtering!

Top 5 Beginner Pitfalls & Traps to Avoid

#	Pitfall	Symptom	The Solution
1	Stale Debounce Timer	Every letter fires a separate OpenStreetMap request.	Always return `() => clearTimeout(timer)` inside your `useEffect`.
2	Passing Numbers as Strings	Backend crashes or MongoDB coordinates become strings.	Always wrap coordinates with `Number(latitude)` and `Number(longitude)`.
3	Missing Auth Token Header	Backend returns `403 Forbidden` when listing produce.	Pass `headers: { 'Authorization': localStorage.getItem('token') }` in `POST /products/add`.
4	Hardcoding API Host	App breaks when deployed live to Vercel/Render.	Always import and use `API_URL` from `../utils`.
5	Category Name Mismatch	Produce is rejected by MongoDB or shelf-life defaults fail.	Ensure the categories in `AddProduce.jsx` exactly match your backend's enum strings!

> 🎉 **Congratulations!**
> With this guide in hand, you have everything needed to construct the complete Veggity frontend. Each piece is modular, easy to understand, and built for real-world speed and freshness. Happy coding!